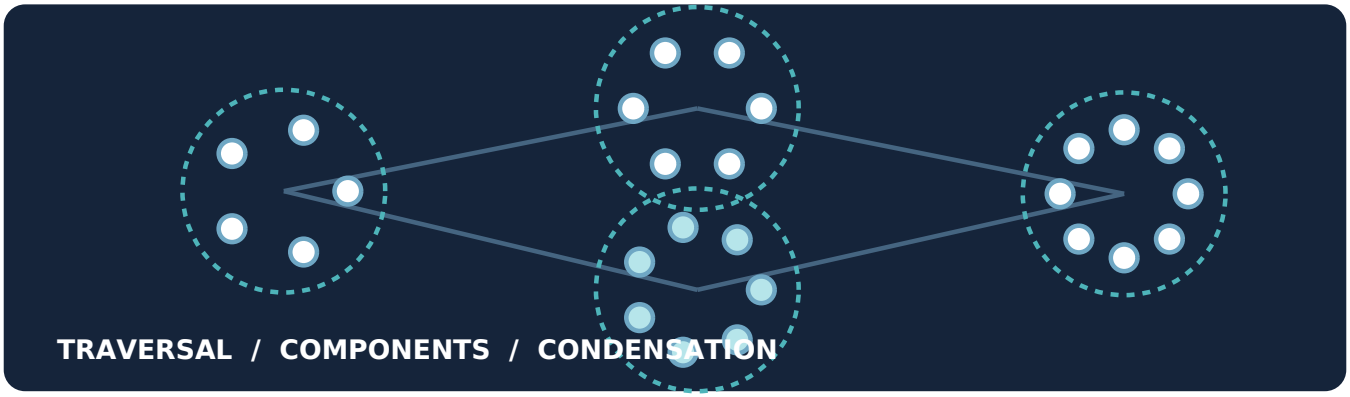
Tarjan’s strongly connected components

A reversible, multi-view directed-graph demo for svg-dom-graph



Decision in one sentence

Use one fixed, deterministic 64-vertex, 114-edge directed graph with 10 strongly connected components; the demo builds its 17-edge condensation DAG incrementally while preserving the original Scene throughout navigation.

64	114	10	17	253
vertices	directed edges	SCCs	DAG arcs	snapshots

Prepared: 4 November 2026 Target: current main-branch demo architecture; crate manifest read at 0.2.47.

Status: revision 6; approved design for Phase 0 and Phase 1, with Phase 2 still gated by native SVG measurements. Follow applies to Play and single steps; SCC landmark jumps centre their destinations, and all navigation reports off-screen attention independently of Follow. User-view changes use `Scene::view`; no new library API is assumed.

Scope boundary

The aim is breadth of functional exercise, not the biggest graph the browser can render. Preserve the 64/114 source topology and verified 253 snapshots; prove legible region-level viewing, bounded non-endpoint connector crossings, a complete scrollable SCC stack, and a zero-node-overlap condensation layout before Phase 2.

1 Intent, scope and size

The demo complements the six-vertex Dijkstra example and the small Edmonds–Karp networks by showing a graph that is large enough for realistic SCC regions and many simultaneous updates. The algorithm lives in pure Rust; the SVG library only displays supplied state.

1.1 Required capabilities

Capability	What must be observable
One state, several views	Every displayed panel is a pure projection of the same indexed snapshot. No panel may run or infer Tarjan independently.
Multiple values per node	Every original vertex displays its index and lowlink, independently of its permanent vertex ID.
Dynamic groups	A closed SCC gains a semantic component assignment and a visible group outline, colour and roster entry.
A genuinely new graph	Each completed SCC is represented as a condensation vertex; a deduplicated directed edge appears only when both endpoint SCCs are known.
Reverse across creation	Going back across a closure removes that group and its condensation representation, while retaining original vertex/edge DOM state.
Deterministic navigation	Previous, Next, Restart, landmark jumps and arbitrary seek have exactly the same outcome regardless of route taken.

1.2 Why 64 vertices is the right maximum

Choose 64 vertices, 114 directed edges for the only large primary fixture. The graph has ten spatial regions: nine non-singleton groups and one singleton. Its chords and cross-component edges reveal the difference between a DFS traversal, Tarjan’s SCC stack, and closed components. The 64-node fixture is a maximum for a readable, pan-and-zoom demo, not a promise that all node labels can be read in a fit-all overview.

- A 32-vertex graph is easier to read but offers less stress for grouping and synchronization; at 64 the challenge is meaningful.
- A 96–128-vertex graph would primarily test label density, zoom and routing clutter. It is better suited to a separate renderer stress benchmark. The 64-node default must pass Phase 0’s measured readability and overlap gates; do not silently reduce the fixture to make them pass.
- Keep a small eight-vertex diagnostic fixture for correctness and multiple DFS roots. Do not add a size slider or a general graph editor.

> **Hard scope limits**
>
> Main fixture: $V = 64$, $E = 114$. Solver budget: $V \le 64$, $E \le 128$, snapshots ≤ 512. The general budget permits the small diagnostic and mutations in tests, not user-supplied arbitrary graphs. The target trace is 252 events and 253 snapshots.

1.3 Explicit exclusions

No graph editing, no runtime force simulation, no generic multi-edge-routing overhaul, no reparenting Scene-owned nodes, no recursive nested Scene per SCC, and no independent clocks. Vertices are non-draggable by default: do not call `make_draggable`. Enable existing source zoom/pan. Phase 0 may freeze an offline-selected coordinate/connector-option table without changing graph topology.

2 View layout and state ownership

A stable desktop stage has the source graph in the upper-left, a vertically scrollable rail holding the complete Tarjan SCC stack and SCC roster in the upper-right, and the independent condensation Scene in a full-width lower strip. All views derive from one snapshot, even though a normal page may scroll vertically on small displays.

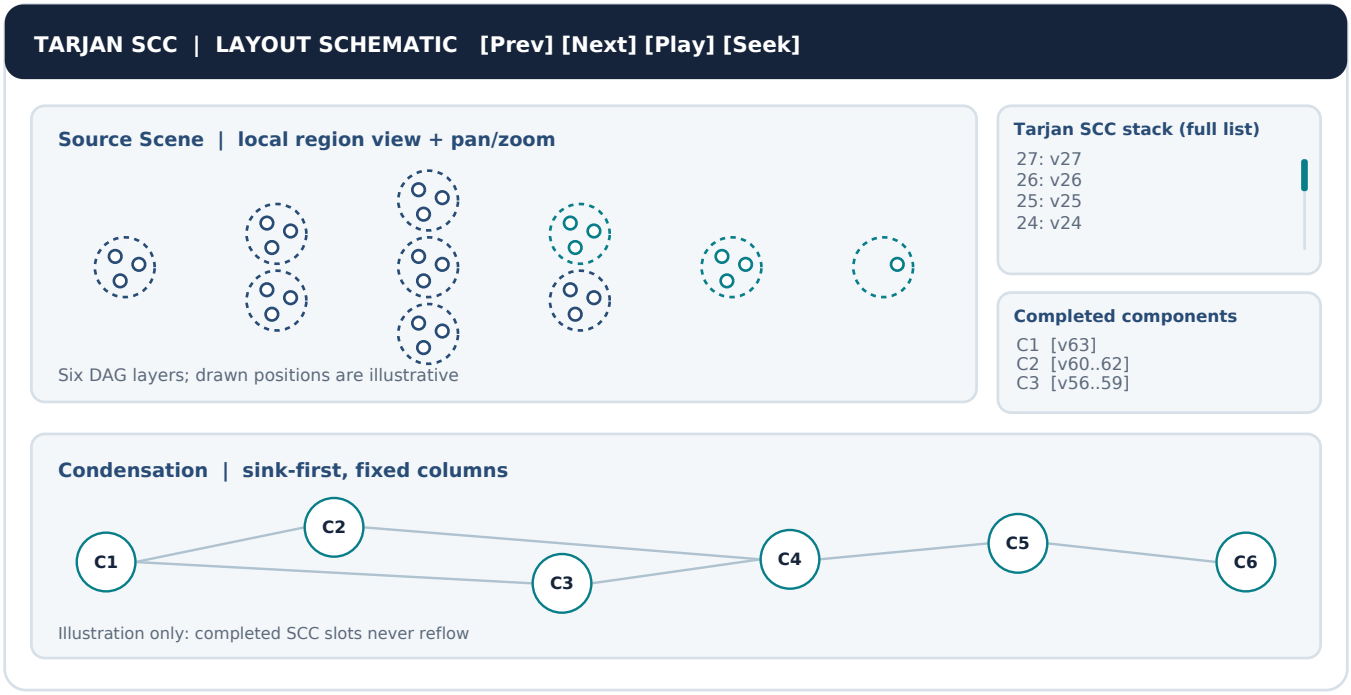


Figure 1. Conceptual multi-view composition (not to scale). The lower condensation view spans the desktop stage rather than being restricted to a narrow right rail; the full stack is scrollable. Geometry is illustrative, not a golden fixture.

Region	Contract
Source / original Scene	Build 64 named data nodes and 114 directed edges exactly once. Its view transform, topology, node positions and DOM identity survive every ordinary step.
Tarjan stack	Render every SCC-stack entry as one DOM row in a scrollable full list (depth max. 27), with clear top and bottom labels; never materialize only the newest 12 rows. This is not the DFS call-frame stack.
Completed SCCs	One immutable row per emitted SCC, keyed by discovery/pop order, including cardinality and vertex IDs; show an outline for the selected row.
Condensation Scene	A separate SVG/Scene in the lower full-width strip, constructed only from completed-component projection. It is empty at snapshot 0 and grows to 10 nodes / 17 arcs.

2.1 DOM and view boundaries

Keep one stable outer stage and fixed controls. The original graph sits in its own long-lived Scene. Insert a host-owned overlay as a child of the Scene’s transformed content group, following the Petri-net/complex-numbers pattern; never mirror its zoom/pan transform. Within this overlay, paint (1) noninteractive component bounding outlines, (2) transparent 114-arc stroke hit paths, and (3) 64 transparent vertex hit rectangles last and therefore above all edge hit paths. A click within a vertex box must select the vertex even when a permitted edge path crosses it. The overlay does not change the Scene’s original node/edge topology. The separate condensation Scene has its own SVG root and view.

Desktop reference: reserve at least 1,120 CSS px for the stage: 20 px left padding + 760 px source pane + 20 px gap + 300 px rail + 20 px right padding = 1,120 px. Source height is at least 720 px; the full-width condensation

strip is at least 320 px high, beneath the controls. At widths below 1,120 px, stack the source pane and the rail vertically rather than squeezing them into a 1,000 px width; retain the lower DAG as a separately scrollable, readable view. Phase 0 measures usable inner sizes, including scrollbars and titles.

2.2 Semantic grouping without connector breakage

When an SCC closes, create a host-owned group record mapping component ID to vertex IDs. Draw a padded bounding outline enclosing the member-node rectangles (no convex hull or concave polygon is implied), with a component label; include membership in the semantic inspector. The outline lives inside the transformed content group and has `pointer-events: none`. Do not reparent Scene-owned nodes or alter their connector topology.

2.3 Stable initial source layout

The former five-column/two-row mini-grid is rejected: with assumed 60×52 and 70×58 node boxes, 62/114 connectors hit a third-party node (30 chords, 17 rings, 15 bridges); an initial ring arrangement still had 39/114. Use the six upstream-to-downstream source-region layers (B0; B1/B2; B3/B4/B5; B6/B7; B8; B9), with a fixed ordered ring of member vertices in each non-singleton region. B9 is one point. Freeze radii, angular offsets, region centers and per-edge native straight/elbow/anchor options in `layout.rs` after Phase 0 measurements; no runtime force layout or new routing API.

2.3.1 Ring-radius feasibility and routing corridors

The ring is a measured spatial structure, not merely a decorative outline. A chord from ring vertex i to i+2 passes the intervening vertex center at distance $R \times (1 - \cos(2\pi/s))$; for s=11 this is about 0.15875R. Clearance of a 32-unit effective half-box therefore already suggests $R \approx 202$ as an optimistic lower bound, not a certified radius. Rotated 60×52 or 70×58 boxes and 4-unit padding can need a substantially larger R; the rectangle support in the chord-normal direction, not just its vertical half-height, controls the real gate. Start the B0 study with R around 210–300 scene units, then measure actual paths and boxes. Reserve a bridge corridor outside each region's measured padded extent; enlarge the source scene as necessary without fitting all 64 nodes at once. The B0 Fit-region ≥0.85 gate remains mandatory.

2.4 Measured source-edge crossing gate

For each of the 114 rendered native connectors, inspect its SVG path in scene-local coordinates against every non-endpoint vertex rectangle expanded by 4 scene units. Record a connector as offending if any point of its sampled path lies inside any expanded non-endpoint rectangle. Use `getTotalLength/getPointAtLength` with sampling gap ≤2 scene units (plus exact straight-segment intersection where possible); count each offending edge once and report its hit vertex IDs. This is a visual-occlusion budget, not a Tarjan algorithm test.

Source measurement	Phase 0 exit criterion
Offending native edges	At most 12 of 114 (10.5%); report ring / chord / bridge breakdown and maximum third-party boxes hit by one edge.
Ring and chord occlusion	Zero ring/chord connectors through a third-party node box in their own source region; prefer changing ring radius/rotation before adding routing variants.
Bounding boxes	No two source-vertex boxes overlap; outline padding at least 8 scene units; validate the original connector IDs and all 114 tip directions.
Path coincidence	No two distinct source connectors may have the same full path after allowing for reversal; the three named antiparallel pairs must have visibly distinct midspans (§2.4.1).
Gate evidence	Save a reproducible table of node geometry, all offending edge IDs and connector options, plus a browser screenshot at first view and the busiest closure.

The 12-edge ceiling is a design acceptance threshold, not a claim already measured on the final layout. If the Phase 0 prototype exceeds either edge gate, adjust only fixed positions and native connector options, rerun

measurement and do not start Phase 2. Do not quietly waive the budget or substitute custom decorative paths for the 114 Scene connectors.

2.4.1 Antiparallel edge disambiguation

Three exact antiparallel pairs arise from the fixture: `56→58 / 58→56` (B7), `60→62 / 62→60` (B8), and `61→62 / 62→61` (B8). A plain straight connector in both directions can render on the same stroke, leaving only the arrowheads distinguishable. Freeze one connector of each pair as a candidate native elbow or give the pair distinct native side/position anchors. Proposed first trial: keep 56→58, 60→62 and 61→62 straight; route the three reverse directions on a distinct elbow/anchor lane. These are candidate settings, not approved measured routes; neither choice may violate the zero internal ring/chord occlusion gate.

Phase 0 compares all 114 sampled rendered paths and fails any complete-path coincidence (including reverse point order; sample maximum pointwise separation ≤1 scene unit). The 14 CSS px non-scaling stroke width of each keyboard/pointer edge hit path requires at least 14 CSS px centreline separation to avoid overlap; use 16 CSS px as the gate (including 2 px tolerance). At scale 0.85, this is 16 / 0.85 ≈ 18.82 scene units (rounded to two decimals). For each named pair, measure the minimum screen-space distance to any point on the opposite complete path, in both directions, at every sample in the central 30–70% arclength of each connector. Require distance ≥16 CSS px throughout that middle 40%, at scale 0.85 and 1.0, and confirm no crossing there. This supersedes the revision-3 ≥8/≥12 scene-unit gate.

In addition to geometry, pointer-test each of the six directed arcs individually with a true hit-test/click at its isolated midspan, asserting the selected stable edge ID, at scale 0.85 and 1.0 and after reversing a closure. Keyboard-test the same six via roving focus and Enter/Space; do not count a focus or click on the opposite arc as success. The 14 px stroke remains accessible, but at manually chosen scales below 0.85 the same disjoint-hit-area guarantee is not required; keyboard selection remains available. Check arrowheads and labels, and record the chosen native connector options and measured distances for each pair.

2.4.2 Congested vertex 62

Vertex 62 has six incident fixture arcs: `55→62`, `60→62`, `61→62`, `62→60`, `62→61` and `62→63`. Its two local antiparallel pairs create a concentrated arrowhead/anchor problem. Phase 0 must photograph vertex 62 at 0.85 and 1.0 after final native routing, inspect arrows against its padded box and labels, and pointer- and keyboard-activate all six incident arcs by stable ID. No arrowhead may terminate inside an unrelated vertex; the 16 CSS px central-pair separation and vertex-over-edge hit priority remain mandatory. If crowded, enlarge B8's ring or adjust fixed native side/position anchors, not the fixture or library router.

2.5 Initial view and fit-region navigation

Do not fit all 64 vertices into the initial source viewport. At snapshot 0, start at 100% zoom (`scale = 1.0`) with B0 / vertex 00 centred; use the actual measured extent of B0 and at least 24 screen-pixel margins. Phase 0 must prove B0 fits at a readable scale ≥0.85; otherwise adjust pane width or fixed geometry before Phase 2. Labels remain legible at both scales and at 200% browser zoom. Source view changes are deliberately limited to the actions in §2.5.1; slider seek is never an implicit pan.

Provide Fit active region and a Follow active vertex toggle, On by default. Fit uses the measured spatial region of the attention vertex, does not rely on the SCC having closed, never uses scale below 0.85 and reports "Region cannot fit legibly" with no view change if impossible. Manual pan/zoom turns Follow Off, but deliberate Prev/Next SCC and Jump to final destination jumps still centre their destination; a jump does not silently turn Follow back On. Seek slider, Restart, Pause and boundary no-ops preserve the viewport. Zoom is never changed by follow or landmark centring.

2.5.1 Attention target and viewport movement policy

Define a deterministic attention vertex for each snapshot. On Enter, InspectEdge or ReturnFromChild use the event's active source vertex; for ComponentClosed use that event's root; on Complete use the most recently closed component's root (v00 / B0 in the reference fixture). The pre-traversal snapshot uses v00. This is an attention anchor, not a new solver field or a claim that the vertex is still on the SCC stack. Its spatial region is obtained from the fixed fixture region map. If a diagnostic trace has no suitable vertex, do not pan and show no active-vertex notice.

Navigation	Source view contract
Play; one Next / Previous	When Follow is On, pan for a changed attention region, or if the attention vertex is outside the 15% inset safe rectangle. With Follow Off, do not auto-pan; the visibility notice still runs.
Next SCC / Previous SCC	Successful jump to a ComponentClosed snapshot centres that newly closed component's spatial region, at current scale, irrespective of Follow. No previous/next landmark: no pan.
Jump to final	Successful jump to snapshot 252 centres B0 / v00, the last closure's region. The DAG is shown separately; source scale is unchanged.
Slider seek; Restart; Pause	Keep current view, including seeks to closure/final snapshots. Recompute a visible off-screen status after the new snapshot or the current view settles.
Fit active region	Explicitly fit the attention vertex's whole region if ≥0.85 is possible; preserves Follow preference.

On eligible Play/Next/Prev, compare the target's fixed spatial region with `last_centered_region`, a controller-owned view-attention marker initially B0. This marker tracks the region the viewport was last deliberately centred on, not the region of the previous Tarjan snapshot. A stationary slider seek never updates it; thus the first Next/Play after seeking to a far-away region correctly recentres rather than merely pushing the vertex to the inset boundary. User pan/zoom invalidates the marker. On a changed region, choose one centring pan: centre the measured padded region bounds if the target vertex will be inside the 15% safe inset; otherwise centre that vertex. Keep scale. Only when the target matches `last_centered_region` may a minimal pan return an attention vertex to the safe inset. Both 34→35 (B0→B3) and 35→34 centre their destination; the trace has 18 changes.

For successful SCC landmark/final jumps, apply the same destination-region centring calculation regardless of Follow. The closure root identifies the destination even when the generic active-vertex field is absent. Update `last_centered_region` only after successful controller pan commit (including Fit); a failed pan or rollback preserves the previous marker. Repeated activation of an already-current landmark is a boundary no-op, not a pan. If a user pan gesture is unsettled, skip competing controller navigation pans, commit the requested snapshot if possible, and show the resulting visibility notice; do not queue a surprise pan after gesture completion. Every controller view change belongs to its snapshot or explicit Fit transaction.

2.5.2 Visible status, even while Follow is On

After every successful navigation and every observed user view change, recompute attention visibility using the committed snapshot and the actual source view, independent of the Follow flag and of which control caused the transition. The safe rectangle is the viewport inset by 15% on each side, in CSS screen coordinates. Outside the safe inset but within the actual viewport, show "Active vertex vNN is near the edge of the view; use Fit active region". Outside the actual viewport, show "Active vertex vNN is off-screen; use Fit active region". Hide the warning only while the attention vertex is inside the safe inset (or absent). The notice has a prominent adjacent Fit action, visible text and matching accessible summary; it is not gated on Follow Off.

For an InspectEdge event, separately determine whether its destination vertex lies inside the actual viewport, not merely the safe inset. Include the target's identity and visibility in the inspector and status, e.g. "Inspect v03 → v33; to v33 (off-screen)". The fixture has 17 inter-region bridges; at a readable local zoom, both ends need not fit. Report the unseen far end rather than suggesting the entire directed edge is visible. A user view-only change updates visibility text without altering the committed Tarjan snapshot; a failed snapshot/view transaction must

not announce its uncommitted target.

2.5.3 Detecting user versus controller view changes

No new `svg-dom-graph` observer is required. Store `last_acknowledged_view` (initially `scene.view()`) and `last_centered_region = B0`. Update the view baseline only after successful controller-owned `set_view` (follow, destination jump or Fit). Compare `scene.view()` before each follow/tick and after native input. Exact equality suffices because readings come from the Scene model. If different, acknowledge the new actual view, invalidate `last_centered_region` and turn Follow Off once. Controller writes never trip the detector. Journal the pre-write view and defer marker changes until Publish.

Capture wheel, pointer-pan, keyboard-pan/zoom and zoom-toolbar input on the stable source host. These signals request a later comparison, not an automatic Follow-Off on mere input. For wheel/key/toolbar actions, reconcile after the native handler (microtask) and after a frame if required. For a pointer gesture, set `pending_user_view_input` at pointerdown on the pan surface, track the pointer ID, and mark it ended on pointerup, pointercancel or lostpointercapture (idempotent; listen where release outside the host is observable). Reconcile after the terminal signal and one animation frame after the last coalesced move; only then clear the guard. If implementation ordering permits a later library frame, extend the checkpoint until its update is observed; assert the order in Phase 0. An unchanged view leaves Follow On. No controller follow/destination pan is allowed while the guard is set. Clear pointer tracking and pending frames on teardown; the guard must never remain stuck after capture loss.

```
reconcile_user_view():
  actual = scene.view()
  if actual != last_acknowledged_view:
    last_acknowledged_view = actual
    last_centered_region = None
    if follow_on: follow_on = false; announce_follow_off()
  refresh_visibility(committed_snapshot, actual)

on_navigation(target, kind):
  reconcile_user_view()   # before deferred tick / follow
  proposed = None; new_center_region = None
  if not pending_user_view_input:
    if kind in [Next, Prev, Play] and follow_on:
      proposed, new_center_region = follow_plan(
        scene.view(), target, last_centered_region)
    elif kind in [PrevSCC, NextSCC, JumpFinal] and is_real_jump:
      proposed, new_center_region = destination_plan(target)
  success = commit_with_optional_view_journal(target, proposed)
  if success and proposed is not None:
    last_acknowledged_view = scene.view()
    if new_center_region is not None:
      last_centered_region = new_center_region
  refresh_visibility(committed_snapshot, scene.view())
```

2.6 Scrollable stack and desktop rail

Render all up to 27 SCC-stack rows in one semantic ordered list, newest vertex at the top, with the visible height capped at 330–390 CSS px and native keyboard-operable scrolling. Never replace the list by a “top 12” summary. Give the rail a separate discovered-components list (all ten rows, independently scrollable if needed); fixed totals and the current-depth line remain outside the scroll area. A user-controlled scroll position is preserved across steps unless an explicit Follow stack top preference is enabled; default that preference on on first load, and turn it off when the user scrolls manually. On reverse, 27 real rows and their order are restored from the selected snapshot, not reconstructed from prior DOM state.

3 Deterministic graph fixture

The source graph is generated entirely in pure Rust from the formula below. No random seed, browser timing, display measurement or HashMap iteration may affect topology or DFS order. These fixture group names are testing labels only; they must never appear as pre-solved SCCs in the runtime UI.

Fixture groups (B0 ... B9)	Vertex IDs	Size
B0	0–10	11
B1	11–20	10
B2	21–29	9
B3	30–37	8
B4	38–44	7
B5	45–50	6
B6	51–55	5
B7	56–59	4
B8	60–62	3
B9	63	1

3.1 Edge construction

```
sizes = [11, 10, 9, 8, 7, 6, 5, 4, 3, 1]
start = prefix_sums(sizes)              # [0,11,21,30,38,45,51,56,60,63]
for each group g of size s >= 2:
    for i in 0..s: add(start[g]+i, start[g]+(i+1)%s)  # cycle
for each group g of size s >= 3:
    for i in (0..s).step_by(2):
        add(start[g]+i, start[g]+(i+2)%s)             # chord
for (a,b) in DAG_PAIRS:
    add(start[a]+(a+b)%sizes[a],
        start[b]+(2*a+b)%sizes[b])                    # one inter-group arc
sort all edges lexicographically by (from, to);
sort every vertex's adjacency list by target ID; reject duplicates.
```

The unique edge total is 63 ring + 34 chord + 17 bridge = 114. The singleton has no self-loop, so no special self-edge routing is needed. An isolated singleton with a self-loop is covered by an algorithm-only micro-test.

3.2 Inter-group DAG (17 arcs)

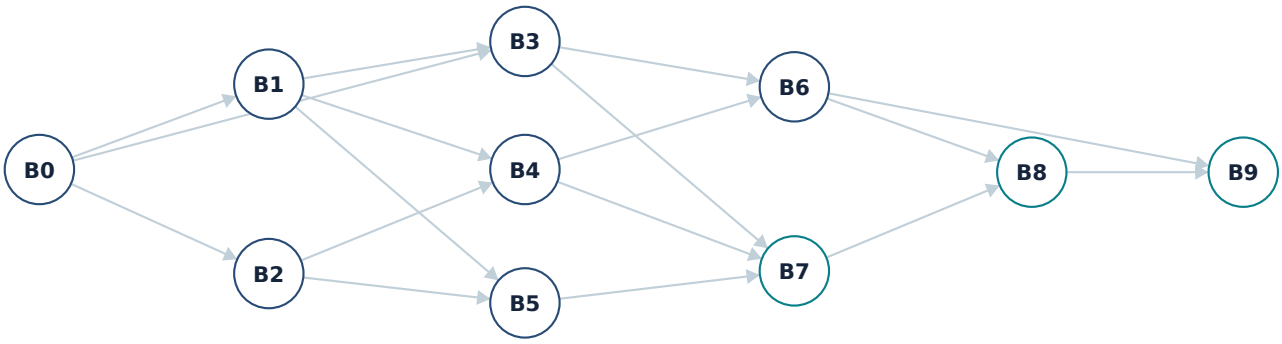


Figure 2. Fixture-generation DAG, labelled B0–B9 only to explain the input. Component labels in the actual demo are assigned by Tarjan in pop order.

```
DAG_PAIRS = [(0,1),(0,2),(0,3), (1,3),(1,4),(1,5),
             (2,4),(2,5), (3,6),(3,7), (4,6),(4,7),
             (5,7), (6,8),(6,9), (7,8), (8,9)]
```

Fixture integrity hash: SHA-256 of the 114 sorted edges serialized as two zero-padded decimal vertex IDs, joined by LF with no trailing LF, each line e.g. `00,01`:

`18ee2b468263021b07a3ed2659a023d6a7c5d3915f1772fb73c5291b776d47eb`

This checksum pins the fixture so refactoring its generator cannot silently change the reference traversal.

4 Tarjan solver: pure, ordered and explicit

4.1 Algorithm state

Field	Meaning and representation
index[v], lowlink[v]	64 optional values in 0…63; both absent before discovery. Counter increments on Enter.
on_stack[v]	64 Booleans. A discovered vertex remains true until its SCC is emitted.
scc_stack	Vector of vertices in push order, the stack drawn by the side panel.
dfs_frames	Explicit call-frame stack `(v, next_neighbor, parent)`; needed to resume traversal, but not shown as SCC stack.
components	Emitted components in pop order, with stable runtime IDs `C1 … C10`.
membership[v]	Optional runtime component ID, populated atomically at ComponentClosed.
cursor / context	Inspected edge, active vertex, optional return edge, event kind and human-readable explanation.

4.2 Update rules (non-negotiable)

1. When entering an undiscovered vertex `v`, set `index[v] = lowlink[v] = next_index`, then increment the counter; push `v` onto the SCC stack and set `on_stack[v]`.
2. For each outgoing arc `v → w`, inspect `w` in ascending ID order. If undiscovered, descend and, on the child's return, use `min(lowlink[v], lowlink[w])`.
3. If `w` is discovered and still on the SCC stack, use `min(lowlink[v], index[w])`. Critically do not use `lowlink[w]` on this branch.
4. If `w` is discovered but no longer on the SCC stack, ignore it for lowlink; its arc remains visible and belongs in the eventual condensation projection.
5. After all outgoing arcs have been examined, if `lowlink[v] == index[v]`, pop through `v`, emitting one SCC atomically. No intermediate "half component" snapshot is allowed.

4.3 Two stacks are different

The DFS frame stack is the execution mechanism. The Tarjan SCC stack is the algorithmic set of discovered vertices not yet assigned to a component; it may contain vertices whose DFS frames have already returned. Labelling both as "DFS stack" would be incorrect. The UI must say "Tarjan SCC stack" and, if the active DFS chain is shown, label it separately.

Correctness invariant

At every snapshot, on_stack[v] is equivalent to exactly one occurrence of v in the SCC stack; emitted components are disjoint; emitted vertices are off-stack forever in the forward trace; and for each discovered vertex lowlink[v] ≤ index[v]. For the large fixture, all 64 vertices are reached from the first DFS root, v00.

5 Trace events and snapshot contract

Precompute the entire event sequence and each post-event snapshot before rendering. Snapshot 0 is the blank pre-traversal state. Event k produces snapshot k. Backward navigation selects an earlier immutable snapshot; it never invokes an inverse Tarjan operation.

Event	Count	Resulting state / presentational meaning
Enter {v}	64	Assign index/lowlink; push the SCC stack; focus v.
InspectEdge {v,w,kind}	114	Focus the directed arc. For stack targets update lowlink using index[w]; classify tree/stack/closed. A tree examination schedules a later Enter.
ReturnFromChild {parent,child}	63	Propagate child lowlink to the parent; mark DFS return and changed value, if any.
ComponentClosed {root,members}	10	Pop all members in one transition; assign C-ID; update group, roster and condensation projection.
Complete	1	Mark end; all vertices assigned; condensation is a 10-vertex 17-edge DAG.
Total	252	253 snapshots, numbered 0…252.

A no-change InspectEdge is still an event, so edges leading to already-completed SCCs are demonstrated rather than silently skipped. Each event stores its operands and old/new values needed for a faithful textual explanation. ComponentClosed includes its stable pop-order member list.

5.1 Verified closure landmarks



Snapshot	New component	Fixture group	Size	Root vertex
74	C1	B9	1	63
77	C2	B8	3	61
81	C3	B7	4	57
104	C4	B6	5	53
117	C5	B3	8	33
167	C6	B5	6	48
198	C7	B4	7	39
206	C8	B2	9	23
247	C9	B1	10	12

Snapshot	New component	Fixture group	Size	Root vertex
251	C10	B0	11	0

All closure snapshot numbers above are fixed assertions against the fully specified fixture and ascending DFS order. At snapshot 252, Complete has fired; no additional SCC is emitted.

5.2 Snapshot materialization

Store `Vec<TraceStep>` of 253 entries, each with the post-event immutable model state. Use a shared backing topology; do not clone 114 rendered edges or DOM nodes into snapshots. At this scale a straightforward snapshot clone is simpler and more reliable than a reversible mutation journal.

6 One projection drives every view

Build a single `project(snapshot, display_preferences)` function that returns all visual and accessible content. It must not inspect DOM state and must not depend on the previously rendered snapshot.

```
TraceStep { event: Event, state: Snapshot }
Presentation {
  source_nodes: [NodeView; 64],   source_edges: [EdgeView; 114],
  stack_slots: Vec<StackRow>,     component_rows: Vec<ComponentRow>,
  group_outlines: Vec<GroupView>, condensation: CondensationModel,
  phase: String, note: String, announcement: String
}
CondensationModel {
  nodes: Vec<{runtime_id, members, size}>,
  arcs: Vec<{from_runtime_id, to_runtime_id}>
}
```

6.1 Original node presentation

Represent each source vertex with a named data node `v00 … v63` and exactly two `NodeValues::Text` cells: index and lowlink. Show `--` before discovery and two-digit decimal strings thereafter; use `DataFormat::Text`, `GridLayout::Columns(2)`, fixed row labels and `with_cell_width_hint("63")`. The string count and type never change, so `set_data_values` can update in place without resizing the node.

The visual distinction between selected, on-stack and emitted vertices must not rely on colour alone. Use existing `set_focus` for current nodes, `set_edge_focus` for the inspected edge, a dashed component outline, and explicit text in the inspector. Mark the component ID in accessibility text and a stable host-owned label; do not modify the numeric values to encode group state.

6.2 Stack and component panels

- SCC stack: render the entire stack as a scrollable ordered list with up to 27 actual rows, current depth and accessible names. Default Follow stack top to on; preserve user scroll if switched off. No top-12 clipping or virtualized rows.
- Component roster: C1…Ck in discovery order with size and members; allow focus/selection of one row to outline its constituent source vertices. Selection is a display preference, not a solver mutation.
- Selected group: if a backward seek deletes the selected component, clear the selection in the same render transaction and report that change.

6.3 Incremental writes and full repair

Compute a minimal patch from the previous successful Presentation, but keep `render_full(presentation)` for dirty repair and equivalence tests. The renderer must clear stale classes, labels, outlines, ARIA attributes and condensation arcs as rigorously as it adds them. No index is committed until all required writes complete successfully.

7 Condensation: a second graph, not a restyle

Construct condensation vertices only from emitted components. For each original arc `u → v`, resolve the two runtime component IDs. Include an arc precisely when both are known and different. Deduplicate component pairs; do not display self-loops or preserve parallel original arcs. Lexicographically sort the resulting runtime-ID pairs.

```
known = { v -> component_id | membership[v].is_some() }
C_nodes = emitted_components_in_pop_order
C_arcs  = sorted_unique({ (known[u], known[v])
    for (u,v) in ORIGINAL_EDGES
    if u in known && v in known && known[u] != known[v] })
# At every step: C_nodes.len() <= 10; C_arcs.len() <= 17.
# At step 252: (C_nodes.len(), C_arcs.len()) == (10,17).
```

7.1 Lifecycle and atomic replacement

Use a distinct Scene hosted by a different nested SVG root, keyed by a condensation signature (ordered emitted component IDs plus sorted arcs). When that signature changes, build the complete new condensation Scene off the live page, with genuine component nodes and directed connectors. Validate it, then replace its host subtree in one commit. When the signature has not changed, do not rebuild: update focus, text and selection only.

Runtime condensation labels are `C1..C10` with size shown in the node or inspector. Use the fixed longest-distance-to-sink column and pop-order slot table of §7.2; neither existing column nor row ever reflows. Place 17 or fewer genuine Scene connectors with tested non-endpoint clearance. Runtime labels are C1..C10 and include size in their visible or semantic name.

7.2 Monotone sink-distance columns (no reflow)

Tarjan emits each completed SCC after every SCC reachable from it in the inter-component DAG has closed. Assign a component its fixed column `d(v)=0` if it has no outgoing condensed arcs, otherwise `d(v)=1+max(d(w))` for its already emitted outgoing targets. These are longest-path distances to a sink, not from a source. Each column is fixed when the component appears; introducing later upstream components cannot change any earlier value. Validate that every outgoing target is already emitted at the closure; an unexpected forward dependency is a solver/projection error.

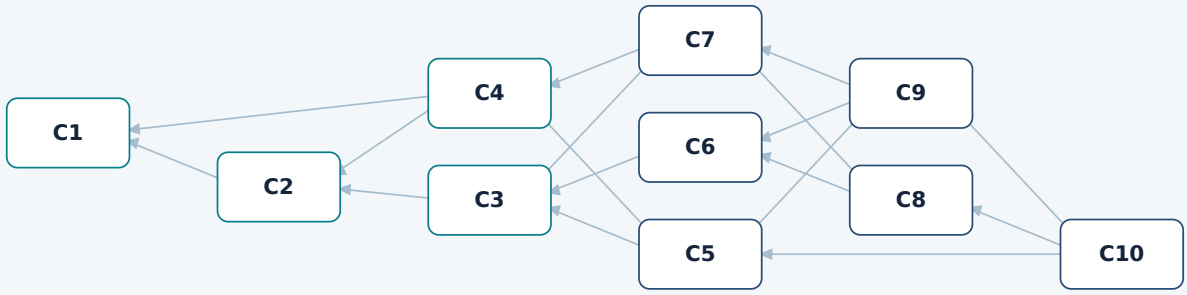
Draw the sink at the left and upstream components progressively to the right. Condensation arrows therefore point right-to-left toward earlier-popped components, opposite to the familiar left-to-right convention. Include the persistent visible legend “Arrows point right to left, toward the sink and previously discovered components” and give the DAG SVG the accessible name “Condensation directed acyclic graph; arrows point right to left toward the sink”. The six completed columns, from left to right, contain 1, 1, 2, 3, 2, 1 components. Within a column, assign the next free row slot in component pop order, not minimum member ID. Earlier node positions are immutable on forward progression; reverse simply removes the last emitted node(s), and revisiting restores the same slots. No 11-signature coordinate table and no reflow algorithm are required.

```
on_component_closed(new_component):
  assert every outgoing condensed target is already emitted
  col = 0 if no outgoing arcs else 1 + max(column[target] for target)
  row = count(emitted components with column == col)  # before inserting
  position = FIXED_SLOT[col][row]
  emit at position; never move any existing component

FIXED_SLOT  # (centre x, centre y), scene units
  col 0: [(100,148)]
  col 1: [(258,103)]
  col 2: [(416,92), (416,181)]
  col 3: [(574,47), (574,136), (574,225)]
  col 4: [(732,92), (732,181)]
  col 5: [(890,47)]
```

Use 96×56 reference collision boxes, 158 column pitch, at least 89 vertical centre pitch within a column, and a 4-unit expansion on non-endpoint boxes. For this specific integer slot table, an offline check of all 17 idealised straight centre-to-centre segments against the expanded third-party boxes reports zero intersections; the minimum centreline-to-expanded-box clearance is about 9.84 scene units. This replaces the incorrect revision-2 B1→B5 candidate. It does not verify actual rendered native connector paths or their endpoints, arrowheads, labels, or measured node sizes.

FINAL CANDIDATE: SINK-LEFT / 6 FIXED COLUMNS / ARROWS TOWARD EARLIER POPS



Idealised straight centre-lines only. Native connector clearance must be measured in Phase 0.

Figure 3. Proposed final condensation graph: C1 is sink-side left and C10 upstream right. Arrows point right to left toward the sink (opposite to conventional left-to-right diagrams). Authoritative coordinates are in the slot table.

Pop order	Group (test only)	Column / row	Proposed centre
C1	B9	0 / 0	(100, 148)
C2	B8	1 / 0	(258, 103)
C3	B7	2 / 0	(416, 92)
C4	B6	2 / 1	(416, 181)
C5	B3	3 / 0	(574, 47)
C6	B5	3 / 1	(574, 136)
C7	B4	3 / 2	(574, 225)
C8	B2	4 / 0	(732, 92)
C9	B1	4 / 1	(732, 181)
C10	B0	5 / 0	(890, 47)

At Phase 0, the live condensation Scene must pass the zero non-endpoint intersection gate with 4-unit-expanded measured node boxes for every nonempty closure stage C1…C10 (and be empty, valid and accessible at snapshot 0). If native connector options or actual box measurements violate the candidate, adjust the ten frozen slot coordinates or per-arc native connector anchors, rerun every stage, and keep prior slots invariant. The 320 px desktop strip and width ≥1,080 px inside the stage accommodate the nominal 96×56 reference layout without shrinking to unreadable text; smaller viewports scroll the condensation scene independently.

7.2.1 Monotonic placement proof and tests

At the j-th closure, the emitted condensation graph is a prefix of the final graph by component-pop order. Each new node has only outgoing arcs to existing nodes; consequently its sink-distance column is defined using already fixed columns, and no existing column changes. The within-column row index is the current count in that column, so that index also never changes. Assert for all 10 closures that the coordinates of all previously emitted nodes and their existing connectors are exactly unchanged; assert the reverse/re-advance round trip gives identical coordinates, connector geometry and semantic labels.

7.3 Reversing through closure

Navigation	Source Scene	Overlay and stack	Condensation
104 → 103	Unchanged objects and layout; restore numeric values/focus from snapshot 103.	Remove C4 membership, hull, roster row; restore its popped vertices to the SCC stack.	Stage the smaller DAG without C4 or its related arcs; swap after successful render.
103 → 104	Update changed values/focus in place.	Recreate C4 overlay/row and pop its members.	Stage DAG with C4 and now-eligible arcs; swap.
252 → 0	Original graph still present, with same 64 nodes and 114 edges, view and IDs.	No groups; every index and lowlink is --; stack empty.	No component nodes or arcs.

Why this is safe

Reverse is snapshot selection, not an attempt to remove a component from a live graph in place. The only Scene replaced is the small condensation Scene. The original graph’s topology, layout and view are persistent.

7.4 Final DAG invariants

Exactly 10 nodes and 17 unique arcs; zero loops, zero duplicate pairs and zero directed cycles (validated by a separate Kahn topological sort). For this fixture, every arc goes from a later-popped component to an earlier-popped component, i.e. `from_runtime_id > to_runtime_id`. All 64 original vertices belong to exactly one component.

8 Navigation, attention and accessibility

8.1 Controls

Control	Required behaviour
Prev / Next	Move one snapshot; with Follow On centre a newly entered region once, else minimally pan for same-region safe-inset departure. Reverse uses identical rules. At a boundary retain view/focus and announce the limit.
Restart	Go to snapshot 0 with identical fixture, never regenerate topology. Preserve source zoom/pan and any display preferences.
Seek slider	Range 0…252; coalesce input to one render per frame and announce on release. Never auto-pan; always show a destination attention warning if outside the safe inset.
Prev / Next SCC	Jump to closest ComponentClosed strictly before/after; centre that closure's region at current scale, even if Follow is Off. No wrap; boundary actions do not pan.
Jump to final	Go to snapshot 252 and centre last-closed B0 / v00 at current source scale; the complete 10-node, 17-arc condensation DAG remains a separate Scene.
Play / Pause	Fixed 800/400/200/100 ms intervals; default 400. Each tick checks for a user view change and commits any eligible follow pan with the snapshot before scheduling the next.
Select Ck	Highlight group in source, roster and condensation without changing the traversal snapshot.
Fit region / Follow active vertex	Fit explicitly, never below 0.85. Follow starts On for Play/individual Next/Prev. Destination jumps centre independently without changing Follow; user view changes turn it Off.

8.2 Epochs and stale work

The one controller owns a monotonic `playback_epoch`, a current snapshot index and a per-render request token. Increment the epoch on every logical stop: pause, boundary press, manual step, seek, speed change, final step, error and teardown. A deferred tick/frame with an older epoch or token is ignored. Cancel outstanding callbacks where possible, but do not rely on cancellation alone.

8.3 Keyboard and screen readers

- All controls use native buttons/range or verified accessible SVG controls; disabled-at-boundary controls stay focusable with `aria-disabled`.
- Provide a text summary of index, lowlink, inspected arc classification, stack depth/top, attention vertex, newest SCC and condensation totals. Announce off-safe/off-screen attention independently of Follow and name an off-screen inspected-arc target. A screen reader must not need colours; all 27 stack entries remain in reading order.
- Graph navigation uses exactly one roving tabindex=0 among 64 host-owned vertex hit rectangles and 114 host-owned stroke hit paths in vertex-ID then sorted-edge order. Host targets live in the original Scene content-group overlay. Paint all edge hit paths first and all vertex hit rectangles afterwards; vertex targets always win overlapping pointer hits, including the ≤12 allowed non-endpoint path crossings. Each edge target copies its native connector geometry, carries a stable `id/data-tarjan-edge`, a descriptive ARIA name, 14-CSS-pixel minimum non-scaling stroke hit width and `pointer-events: stroke`. Vertex rectangles cover the measured node boxes and carry stable vertex IDs. Enter/Space inspects without advancing; `set_focus` and `set_edge_focus` supply visible attention. Prevent native SVG node tab stops from making a second roving target. Verify copied edge path geometry after initial build; source nodes do not drag, so it remains fixed.

- A manual navigation change announces snapshot number plus event. During playback, announce SCC closures and the final DAG, not all 114 arc inspections. The Follow active vertex toggle has stable On/Off text and `aria-pressed`; actual user pan/zoom announces “Follow active vertex off”. Regardless of Follow, outside-safe attention and off-screen inspected endpoints are named in visible and accessible status, with a prominent Fit-region remedy.
- On a condensation rebuild, preserve external control focus. If focus was inside a disappearing component, move it to a stable condensation title/summary or component roster target, never to removed DOM. Maintain the right-to-left arrow legend and DAG accessible name even while the DAG is empty.

Phase 0 must verify that exactly one source vertex/edge hit target has `tabindex=0`, that a user can reach every one of the 114 edge targets using the documented roving keys, and that hit paths preserve identities and coordinates through pan, zoom, snapshot reverse and focus changes. In particular, pointer hit-test and keyboard-activate each of the six distinct antiparallel arc targets at scales 0.85 and 1.0. If native node focusability conflicts with roving tabindex, fix the host coordinator before Phase 2.

8.4 Appearance and reduced motion

Use an SCC colour palette that repeats with a second encoding: dashed hulls, component labels and roster membership. No animated node movement is necessary; honour `prefers-reduced-motion` by suppressing transition effects. The graph must remain interpretable at 200% browser zoom and with colour removed.

9 Commit protocol and failure recovery

The primary risk is a partly updated multi-view page whose panels disagree about which snapshot is current. Treat each snapshot render as a commit with a single published index, and ensure the recovery route is deterministic.

1. Plan. Reconcile `scene.view()` against the acknowledged controller baseline before Play, a step, a landmark jump or a pending tick. Compute the target Presentation and its attention vertex; select a region-centred / minimal follow pan or a destination-centred landmark pan only if allowed by §2.5.1. Stage condensation off-page. Save the actual current view for rollback, not a stale baseline.
2. Write. Journal source values/focus, group overlay, stack/roster, inspector, `a11y` and any permitted Follow or landmark destination pan in one target transaction. An unsettled user pan suppresses any controller pan. Failed pan restores the saved view or marks dirty; the snapshot index stays old until Publish.
3. Swap. If the condensation signature changed, replace its staged subtree atomically. Journal the previous subtree and live attributes so a failed later write can be restored.
4. Publish. Only after all writes succeed, publish index/presentation; advance `last_acknowledged_view` and any `last_centered_region` only for a committed controller pan, then derive visibility/status from the committed snapshot and actual source view. Never publish the target’s off-screen notice or change controller markers after a failed/rolled-back pan.

If any write fails, stop playback, invalidate pending callbacks and mark the page dirty. Retain the last committed snapshot. On the very next user action, including a boundary no-op or selecting the already-selected component, attempt a full render of the last committed snapshot first. If repair fails, do not process the requested action. A failed build before any live DOM change need not dirty the live page.

A failed condensation swap must leave the previous graph attached; a failed rollback still marks dirty and is repaired from a full snapshot. Keep component Scene generations unique, so a staged node cannot clash with live SVG IDs. Do not report success or update the slider value until the committed snapshot matches every panel.

9.1 Persistence rules

- Every navigation preserves original Scene identity, 64 NodeIds / 114 EdgeIds and vertex positions. Source view changes only for eligible Play/Next/Prev Follow pans, successful SCC/final destination-centred jumps, explicit Fit, or genuine user pan/zoom. Seek slider, Restart, Pause and boundary actions preserve the viewport. A controller pan does not re-enable Follow after a user override.
- A condensation rebuild preserves its own zoom/pan if the node count remains nonzero, unless the user selects “Fit DAG”; the source view is independent.
- On teardown: invalidate epoch/token; cancel timers/frames; detach listeners and observers; discard staged SVG; clear callbacks and debug hooks.
- No callback is permitted to retain a stale generation and mutate a newer Scene.

9.2 Required test hooks

Provide test-only timers/frames, synthetic pan/zoom and fault injection for value, overlay, Follow `set_view`, landmark destination pan and condensation replacement. Hooks are excluded from production. Check stale ticks after Pause→Play, user view changes, seek, failures and teardown; include failed Next and failed SCC-jump pan followed by full repair, and a lostpointercapture after coalesced pointer moves.

10 Tests and measurable acceptance

10.1 Algorithm unit tests: native Rust

Test area	Non-negotiable assertion
Fixture	64 unique IDs, 114 unique arcs, exactly 17 DAG_PAIRS, SHA-256 matches §3; all sorted adjacency lists.
Oracle	Independent Kosaraju SCC partition equals Tarjan’s partition for the 64-node fixture and exhaustive small directed graphs up to four vertices; random graphs through 12 vertices.
Lowlink cases	Tree propagation uses child lowlink; on-stack arcs use target index; arcs to closed vertices do not update lowlink; isolated and self-loop singletons tested.
Event trace	64 Enter + 114 InspectEdge + 63 ReturnFromChild + 10 ComponentClosed + 1 Complete = 252 events / 253 snapshots.
Landmarks	All ten closure positions/root vertices in §5.1; pop sizes [1,3,4,5,8,6,7,9,10,11].
Invariants	Every snapshot has consistent on-stack bits, no duplicate membership, lowlink bounds, and identical covered vertices between stack and emitted SCCs where applicable.
Condensation	Projection at every snapshot has no loops/duplicates/cycles; final graph contains exactly 10 nodes and 17 arcs.
Negative controls	For arcs 0→1, 0→3, 1→2, 2→0, 3→1: after inspecting 3→1, lowlink[3] must be 1 (incorrect lowlink[1] gives 0). For only 1→0: ignoring the previously closed v0 is essential to emit {1}.

10.2 Projection, DOM and navigation tests

Test area	Non-negotiable assertion
One snapshot, four views	Source, stack, SCC roster, hulls, inspector and DAG all have one matching snapshot ID after arbitrary seeks.
Forward / backward	For every k=0…252, rendering k→k+1→k produces the same semantic DOM state as a fresh full render of k.
Creation reversal	Seek 103→104→103 and 250→251→250; no stale group, caption, C-ID or DAG arc survives.
No source rebuild	Original Scene root, source NodeId/EdgeId mapping and positions remain stable. View changes are limited to explicit Fit, user pan/zoom and eligible committed follow transitions.
Geometry / crossings	Source ≤12 of 114 offending native edges and zero same-region ring/chord node hits. All ten nonempty condensation stages have zero native non-endpoint node-box hits; empty stage is valid.
Initial readability	Snapshot 0 centres B0 at scale 1.0 with readable fit ≥0.85. Play and one-step Next/Prev Follow centre on region changes (34→35 and reverse), with minimal pan only within a region. Slider seeks stay stationary; landmark jumps centre destinations.
Full stack	Stack peak is 27; all 27 rows are present and keyboard-scrollable at peak; reversible order and user scroll/follow policy at every snapshot.
Hit layering	114 edge stroke-hit paths below 64 vertex rectangles; no edge steals vertex clicks. Test individually clicking each of the six antiparallel targets at both 0.85 and 1.0 (and after closure reversal), and Enter/Space on each distinct roving target.
Antiparallel paths	All 114 paths are noncoincident. Six arcs in three antiparallel pairs pass ≥16 CSS px central separation at 0.85 and 1.0, plus individual pointer/keyboard activation. Check all six arcs incident on vertex 62 for arrowhead crowding and hit priority.
DAG monotonicity	Every closed C-ID receives a sink-distance column and pop-order row on emission; every earlier component retains identical coordinates and visible connector paths across all ten closures.
Diff = full	For every snapshot, diffed DOM state agrees with full projection, including removed classes, ARIA attributes, stale edges and outlines.
Cross-signature seeking	At all ten closures seek before→after→before and after→0→after, then compare diffed vs full DAG DOM, immutable slot coordinates, focus and ARIA; include 252→74→251→0.
Failure injection	Before and after a failed swap, invalid DOM writes stop playback; dirty repair precedes next action, even when at a boundary.
Stale scheduling	Old ticks/frames after stop/speed/seek/teardown cannot alter state. Seek coalesces input. No controller pan survives its own transaction. Pointerup, pointercancel or lostpointercapture plus a settled post-move frame clears gesture guard; late user pan wins.
Follow movement	Follow starts On. Play and single Next/Prev centre a new viewport-attention region once, then minimally pan only within that centred region. 34→35 and 35→34 centre their destinations; Follow Off suppresses these pans. Scale stays fixed.
Landmark vs seek	Next SCC 0→74 centres B9 even with Follow Off; Previous SCC 77→74 centres B9; Jump final centres B0. Slider seek 0→74 and 252→74 stays stationary and shows attention notice. After seek 0→74, single Next with Follow On centres B9, not inset-boundary pan.
Off-screen status	For every navigation and observed user pan, outside 15% safe inset shows visible and accessible status regardless Follow. Distinguish near-edge from truly off-screen. Cross-region InspectEdge reports off-screen destination, e.g. to v33 (off-screen).

Test area	Non-negotiable assertion
User-view detection	Wheel, background drag, key pan and zoom toolbar toggle Follow Off only on actual Scene.view delta; Follow/Fit/landmark controller writes never do. A native pan between ticks wins. Verify pointerup, pointercancel and lostpointercapture (including release outside host) after coalesced moves.
Accessibility	Single roving graph target, stable right-to-left DAG legend/accessibility name, predictable focus on rebuilt DAG, live region semantics, contrast and reduced motion.

10.3 Performance and resource gates

Profile the release demo on a normal desktop browser, not through debug builds. Measure median and p95 of node-value/focus projection and the ten condensation rebuilds separately. Aim for p95 < 32 ms for an ordinary step and a responsive seek; do not make hardware-specific timing a mandatory CI assertion. Also assert exact resource ceilings: 64 original nodes, 114 original edges, 10 final condensed nodes, 17 final condensed arcs and ≤512 snapshots.

Keep the original graph DOM stable to avoid repeated 64-node/114-edge rebuilds on each step. Compare a fast forward walk to arbitrary seeking; failures of functional equivalence are blockers, while microsecond-level timing differences are not.

11 Implementation plan and phase gates

Phase	Deliverable	Exit gate
0 — API/layout proof	Measure source rings, bridge corridors and native paths; prove ≤12 source non-endpoint hits, ≥16 CSS px antiparallel central separation, independent targets, vertex 62 arrows, hit priority, readable B0, full 27-row stack, zero DAG hits. Prototype Scene.view reconciliation on native wheel/drag/key/toolbar changes.	Block Phase 2 on all §2 / §7.2 geometry and hit-target gates; confirm view reconciliation works with coalesced dragging. Preserve Phase 1 regardless.
1 — solver + fixture	Pure Rust graph generator, explicit DFS frames, snapshots, Kosaraju oracle, closure landmarks, hash and negative-control fixtures.	Approved to start now; all native algorithm tests pass and solver has zero DOM dependencies.
2 — source + panels	Persistent 64-node/114-native-edge Scene, text cells, 27-row scrollable stack, roster, descriptions and manual stepping; only after Phase 0 geometry sign-off.	Phase 0 gate recorded and green, including antiparallel separation and hit priority; stable source DOM; snapshots 0/74/104/251/252 match golden presentations.
3 — grouping + DAG	Content-group bounding outlines, a staged condensation Scene using immutable sink-distance slots and reversible DAG focus.	Ten nonempty closure states pass zero-crossing gate and immutable-position tests; empty state is valid; reversal preserves original Scene IDs/view.
4 — controller + robustness	Seek, follow on Play/single steps, landmark centring, unconditional visibility status, settled-gesture reconciliation, epoch/token, repair, swap, a11y and lifecycle.	Diff/full, region-entry centring, jump/seek contrast, off-screen arc text, capture-loss, failed-pan rollback and stale-callback tests pass.
5 — visual/perf polish	CDP visual audit where stable, manual browser verification, release profiling, documentation and demo gallery entry.	Acceptance matrix green; no functional regression in other demos.

11.1 Repo placement and public surface

Keep this implementation demo-local under `demo-app/src/tarjan/`, with separate `algorithm/`, `fixture.rs`, `trace.rs`, `projection.rs`, `render.rs`, `controller.rs` and `layout.rs` as complexity justifies. Add one gallery panel and one demonstration documentation page. Do not change the crate's public API merely to support this one fixture.

11.2 Phase 0 checks grounded in current repo

- `Scene::add_named_data_node` and `Scene::set_data_values` are documented for fixed-count updates.
- `NodeValues::Text`, `DataFormat::Text` and `with_cell_width_hint` support pre-discovery "--" without resize.
- `Scene::set_focus`, `Scene::set_edge_focus`, `view` and `set_view` are documented. The host overlay must attach to `.svg-dom-graph-content` and inherit transform; test actual hit target geometry and atomic swap.
- Existing Edmonds–Karp and N-Queens demos provide controller, full-repair, coalesced seek and accessibility patterns. Reuse established patterns instead of inventing another scheduler.

Build readiness criteria

Phase 1 remains approved independently of layout. Phase 2 still needs browser evidence for (1) ≤12/114 source crossings and zero internal ring/chord through-node hits, (2) B0 fit ≥0.85, (3) full 27-row stack, (4) zero condensed node-box hits in ten nonempty stages, (5) vertex targets above edges, (6) 114 distinct paths and all six antiparallel arcs reachable at 0.85/1.0 with ≥16 CSS px separation, including vertex 62, (7) fixed DAG slots, and (8) `Scene::view` reconciliation including capture-loss settlement. Navigation/status changes are controller requirements and do not alter the solver fixture.

12 Revision 2 → 3 decision record (retained)

Revision-2 review issue	Revision-3 response / verification
1. False analytic zero in DAG	Replace source-distance candidate with ten sink-distance slots (x = 100 + 158d); offline geometry test of all 17 straight centre-lines against 96×56 boxes plus 4 gives zero hits, ~9.84 minimum clearance. Browser gate remains mandatory.
2. Three coincident pairs	Name the exact antiparallel pairs 56↔58, 60↔62 and 61↔62; use tested native elbow/anchor variants and a quantitative midspan separation gate.
3. Avoidable DAG reflow	Longer path to sink determined entirely from already completed successors; pop-order append slots mean not one previously revealed vertex changes position; one ten-slot table, not eleven layouts.
4. Large ring sizing	Use chord clearance formula as a lower bound, plan B0 radius around 210–300 units and measured bridge corridors; never assert an unmeasured source layout clears.
5. Hit priority and panes	Layer 64 vertex targets above the 114 edge targets; increase desktop stage to 1,120 px with explicit 20/760/20/300/20 allocation; narrow stage stacks vertically.
6. Empty condensation state	Zero closure means no DAG layout to measure; validate empty representation separately, and inspect the ten nonempty stages.

Release decision: Phase 1 remains approved. Phase 2 remains gated by the actual browser measurements for the source, three antiparallel paths, node-over-edge pointer priority, and the ten condensation states. The verified fixture SHA-256, 252 events, 253 snapshots, closure indices, SCC partition and 17 arcs are unchanged.

13 Revision 3 → 4 decisions

Revision-3 review point	Revision-4 contract
Playback action off-screen	Follow during Play starts On, only runs during Play, and pans at region changes / departures from a 15% safe inset. Manual pan/zoom immediately turns it Off and keeps it Off through Pause, Restart and later Play until explicitly restored.
Hit width vs separation	The old 8/12-unit midspan threshold is replaced by ≥16 CSS px centreline separation for each antiparallel direction across the central 40% at scales 0.85 and 1.0. All six edges require independent pointer and keyboard activation tests.
Right-to-left direction	The final DAG remains sink-left with right-to-left arrows. A persistent visible legend and accessible DAG name state this nonstandard direction, even at the empty stage.
Unchanged construction	The ten frozen sink-distance slots, 17-arc final DAG, and atomic condensation Scene replacement across closure and reverse events are unchanged.

Approval: Revision 3 was approved for Phase 0 and Phase 1. Revision 4 changes only user navigation, edge-hit disambiguation gates and direction labelling. Phase 2 still depends on live Phase 0 source and DAG measurements, not analytic predictions. The fixture checksum, SCC partition, closure snapshots and 253-snapshot trace remain fixed.

14 Revision 4 → 5 decisions

Revision-4 review point	Revision-5 contract
Manual Next/Prev goes off-screen	The same follow preference applies to Play and individual Next/Prev. At 34→35 (and 35→34), follow the changed active region; seek, landmark jumps, Restart, Pause and boundary actions remain stationary.
No view-change observer	Read Scene.view against last_acknowledged_view before follow/tick and after native inputs. Store controller-owned set_view results as the new baseline; guard coalesced user gestures and never disable follow for a controller pan.
Arrow crowding at vertex 62	Audit all six incident directed arcs, arrowheads, native connector anchoring and click/keyboard targeting at scales 0.85 and 1.0. Keep the ≥16 CSS px antiparallel midspan gate.
Numerical rounding	At 0.85, 16 CSS px is 16/0.85 = 18.8235… scene units, or 18.82 to two decimal places.

Approval: Revision 4 was approved for the existing Phase 0 and Phase 1 work. Revision 5 only resolves navigation visibility and view-change observation and expands the Phase 0 vertex-62 audit. Phase 2 remains gated by real browser geometry; fixture hash, 252 events, 253 snapshots and condensation layout are unchanged.

15 Revision 5 → 6 decisions

Revision-5 review finding	Revision-6 contract
Seek and SCC jumps can hide the action	The off-safe/off-screen notice runs after all navigation and user-view changes regardless of Follow. Slider seeks stay stationary and do not change last_centered_region. Successful SCC jumps centre closure regions; Jump final centres last-closed B0 regardless of Follow, without changing its setting.
Smallest pan leaves most of new region off-screen	On a changed region, choose one region-centred pan, or attention-vertex-centred fallback when the vertex would miss the 15% safe inset. Only within the same region is the pan minimal. All preserve scale.
Inter-region inspected edge is not fully visible	Show inspected target ID with “(off-screen)” when the target lies outside the actual viewport, even if the source is visible. All 17 bridge arcs are covered by the general rule.
Unsettled user drag / capture loss	Pointerup, pointercancel and lostpointercapture end a guarded pointer gesture; reconcile after the final coalesced move frame, then clear guard. Capture loss and teardown must never leave a permanent pending state.

Approval: The revision-5 solver, 114-edge fixture, trace, closure landmarks, DAG slots and geometry gates are unchanged. Revision 6 refines only navigation attention, view reconciliation, and status/accessibility. Phase 0 and Phase 1 remain approved to proceed; Phase 2 remains gated by real browser geometry and hit-target measurements.

Appendix A Minimal reference solver pseudocode

The implementation may be iterative, as recommended to expose explicit DFS frames, or use an equivalent bounded recursive builder. In either case the emitted event order must be identical. The following describes the pure state changes, not the event-logging plumbing:

```
next_index = 0; scc_stack = []; components = []
for v in 0..64:
  if index[v] is absent:
    visit(v)

visit(v):
  index[v] = next_index; low[v] = next_index; next_index += 1
  scc_stack.push(v); on_stack[v] = true; emit Enter(v)
  for w in sorted_adjacency[v]:
    kind = tree if index[w] absent else
           stack if on_stack[w] else closed
    if kind == stack: low[v] = min(low[v], index[w])
    emit InspectEdge(v, w, kind)
    if kind == tree:
      visit(w)
      low[v] = min(low[v], low[w])
      emit ReturnFromChild(v, w)
  if low[v] == index[v]:
    members = pop_through(scc_stack, v)  # atomic closure
    for w in members: on_stack[w] = false
    components.push(members)
    emit ComponentClosed(v, members)
emit Complete
```

For an iterative implementation, the frame records which successor to inspect next and an optional parent; Emit/Return/Close boundaries must remain exactly as above. The presentation model carries one event per snapshot, and recorded snapshots are immutable after construction.

Appendix B Additional micro-test fixture

Use eight vertices with SCCs {0,1,2}, {3,4}, {5,6} and {7}, with cycles 0→1→2→0, 3↔4 and 5↔6, bridges 2→3 and 4→5, and vertex 7 disconnected. This checks a second DFS root and a singleton; a separate one-vertex 0→0 test checks the solver's self-loop semantics without requiring Scene to render a self-connector.

Pin two mutation sentinels: (a) edges 0→1, 0→3, 1→2, 2→0, 3→1 give lowlink[3]=1 on examining 3→1; using the already-reduced lowlink of 1 instead would give 0 even if the final SCC partition happens to match. (b) a graph with only 1→0 must emit two singleton SCCs, rather than lowering vertex 1 through the already-closed vertex 0.

Appendix C Sources and API references

Tarjan, R. E. (1972). "Depth-First Search and Linear Graph Algorithms." SIAM Journal on Computing, 1(2), 146–160. DOI: 10.1137/0201010.

Project: github.com/ChrisWhealy/svg-dom-graph; manifest inspected on 4 November 2026 at version 0.2.47.

Library API: nodes.md, text-cells.md, selection.md, connectors.md and toolbar-and-zoom.md.

Related demo patterns: Edmonds–Karp, Dijkstra and N-Queens.

Final acceptance statement

At snapshot 252 the original directed graph still has 64 vertices and 114 edges; all 64 vertices belong to 10 SCCs; the condensation Scene has 10 nodes and 17 unique acyclic arcs. Every previous snapshot—including one before any SCC—can be revisited without rebuilding or losing the source Scene and its IDs. Only explicitly specified Follow, landmark destination, Fit or user view actions change its viewport; visibility feedback always describes the committed state.